

Thryv Mobility

Internship Report



XRIM

Documentation

Documented by: Mugdha Meda

Duration of internship: 27 May, 2024 - 27 July, 2024

INDEX

1. Simulation of Wheel Motion on Gazebo: (Software Pipeline).....	2
1. ROS2, TURTLESIM, RVIZ, GAZEBO: UPSKILLING.....	2
2. GENERATING URDFs: SbW and XRM's Test Rig.....	2
3. GENERATING LAUNCH FILES & VELOCITY PUBLISHERS.....	3
2. Simulation of Wheel Motion on Gazebo: (Software - Hardware Pipeline).....	4
1. ARDUINO IDE TO EMBED uROS ONTO ESP.....	4
2. ESP-IDF FOR ROBUSTNESS: ESP-IDF + uROS.....	4
3. VESC-ESP32 SETUP & SCRIPTS.....	5
3. Controller Scripts for Adaptive Resistance Braking in XRM.....	6
4. Encoder Mount and Coupling for existing XRM Test Rig and Testing Data Retrieval.....	7
5. Additional Documentation:.....	8
1. Commands, Resources for Easy Use of ROS2, Gazebo, uROS, ESP-IDF, VESC tool.....	8
2. Kadham Knee Pin Surface Quality Documentation.....	8
3. Work Completed (Zip Files).....	8

1. Simulation of Wheel Motion on Gazebo: (Software Pipeline)

1. ROS2, TURTLESIM, RVIZ, GAZEBO: UPSKILLING

(May 27 to June 12)

- On installing Ros2, the first 2 weeks were spent learning to use ROS's nodes, topics, messages etc, from the official site's documentation. Wrote publisher and subscriber nodes to test understanding, learnt about CMakeLists and Colcon Build.
- Wrote scripts in C++ for practice to control the velocities of the turtlebot in turtlesim to draw figures- circles, eights, etc.
- Familiarized myself with Rviz and Gazebo's environment, supplied velocity commands to test models consisting of obstacle collisions and sensors like lidar.
- [RESULTS](#): Now capable of writing scripts for message publishing and subscribing and writing my own nodes, as well as familiar with Gazebo and Rviz.

2. GENERATING URDFs: SbW and XRM's Test Rig

(June 12 to June 14)

- Installed sw2urdfSetup.exe (from GitHub) and Solidworks to create URDFs for the CAD models of the existing test rig and wheelchair.
- Created Coordinate Axes, Axis(for every revolute joint) on every part involved in the URDF parent-child tree to generate the URDF from the add-on.
- **ISSUES FACED:**
 - (i) The model would lie on its sides in Gazebo. (z axis oriented wrong with respect to the model)
 - (ii) Issues with the physics of the model, i.e. unstable steer of caster wheels and drifting of wheelchair
- **RESOLVED:**
 - (i) Re-oriented the y-axis of the model in Solidworks to ensure the model lands upright in Gazebo.
 - (ii) provided appropriate friction for the revolute joints in the URDF to mimic real life conditions (vary friction and damping parameters)
- [RESULTS](#): URDFs of Wheelchair and Test Rig successfully made, and physically sound simulation observed.

3. GENERATING LAUNCH FILES & VELOCITY PUBLISHERS

(June 14 to June 17)

- Generated XACRO files for simplified maintenance- to make the URDF code more modular and easier to re-use. SbW and XRM Testrig XACROs were written.
- Meshes were imported from the CAD models and Launch files were generated to include inter-related files and Controller Parameters for ROS2. Examples files were provided to assist in learning and have been included in the material folder for future help.
- C++ ROS2 velocity scripts to control simulated objects in Gazebo were written, and final simulated movement was verified (minimum phase steer and right orientation of caster-given forward or backward velocities to the drive wheels.)
- [RESULTS](#): Generated the software end for the simulation of physical motion of the wheels of the test rig, to be connected through the micro-ros agent to the esp32.



2. Simulation of Wheel Motion on Gazebo: (Software - Hardware Pipeline)

(ROS2 <-> uROS <-> uC(ESP32S3) <-> VESC <-> Motor)

1. ARDUINO IDE TO EMBED uROS ONTO ESP

(June 18 to June 22)

- Set up a Micro-ROS agent and learnt to write scripts for Micro-ROS and ROS2 interaction (publisher, subscriber ,etc) using arduino IDE.
- **RESULTS:** Successfully connected uROS on the ESP32 to ROS2, receiving and sending data, and generating scripts to send data received from VESC to the required node in ROS for simulation in Gazebo. Was advised to shift from Arduino IDE to ESP IDF for more robust handling.

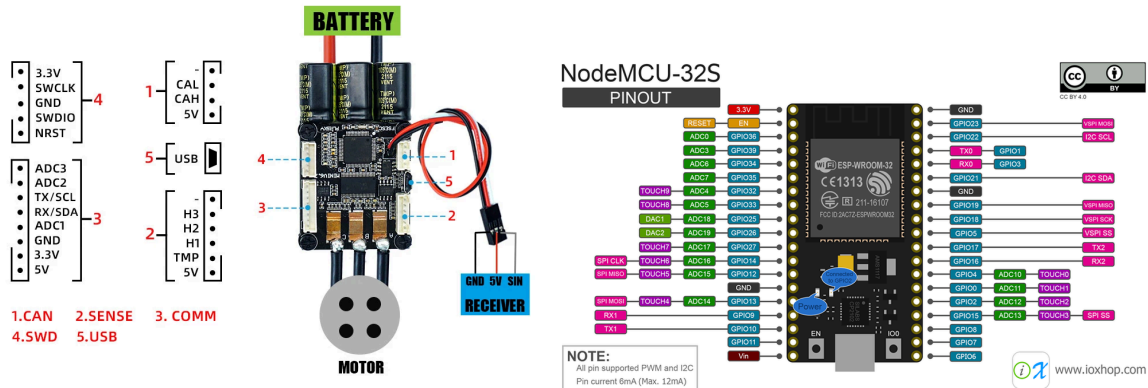
2. ESP-IDF FOR ROBUSTNESS: ESP-IDF + uROS

(June 24 to July 4)

- Installed ESP-IDF and wrote code for simple testing with esp32 and esp32s3.
 - Tried out 3 different esp-idf and uROS integration methods. 2 of higher level interfaces had issues such as only working for esp32, and not other esp boards,etc.
 - Utilized the official lower level package: micro_ros_espidf_component.
 - **ISSUES:**
 - (i) After much debugging, the official package seemed to work, but without the UART functionality. Connection was only established through wifi. Example scripts worked over wifi. Tried re-installing multiple times, ensured dependencies,etc, but that does not seem to be the issue.
 - (ii) After much usage of the wifi functionality, even this provision began to fail. Unable to resolve the error:
`I (4835) wifi:<ba-add>idx:1 (ifx:0, ce:84:3d:81:c8:db), tid:6, ssn:4, winSize:64`
`Failed status on line 54: 1. Aborting.`
The micro_ros_espidf_component seems to have worked for many other users despite being a newer provision. Failed to resolve this issue, could not run scripts written to connect esp32 to ROS through uROS.
- RESULT:** Was advised to move on to the rest of the work given that this issue was taking too long to resolve. This will be looked into by someone with more experience with esp-idf.

3. VESC-ESP32 SETUP & SCRIPTS

(June 24 to July 4)



- Installed the VESC tool, connected the VESC to the esp and tested receiving and sending data successfully. Spent time understanding different parameters in the VESC tool and their physical
 - ISSUES:**
 - (i) FOC does not work for the single wheel setup present. Tested with all 4 VESCs, and only Brushless Mode works for this hub motor. FOC functions just fine for the scooter board motor, pointing to faults with the GoGoA1 motors.
- RESULTS:** Worked with Brushless configuration for the GoGoA1 hub motors, sent data to the esp and was able to display it and send it through micro-ROS to ROS through arduino IDE



```

getVescValues.ino
58
59 /** Setup UART port (Serial1 on Atmega32u4) */
60 // Initialize Serial1 for VESC communication at 19200 baud rate
61 // Using pins 16 (ESP32 RX) and 17 (ESP32 TX)
62 Serial1.begin(115200, SERIAL_BN1, 16, 17);
63
64 while (!Serial) {}
65
66 Serial.println("Serial port initialized!");
67
68 /** Define which ports to use as UART */
69
Output: Serial Monitor X
Message (Enter to send message to 'ESP32-WROOM-DA Module' on '9devityUSB')
u.00
35.90
0.00
3430
Data received from VESC:
0.00
35.90
0.00
3430
Data received from VESC:
0.00
35.90
0.00
3430
Data received from VESC:
1744.00
35.90
0.00
3470
Data received from VESC:
1640.00
35.90
0.00
3648
Data received from VESC:
1250.00
35.90
0.00
3789
  
```

3. Controller Scripts for Adaptive Resistance Braking in XRIM

(BrakeCurrent = $a_1(\text{theta} - \text{mean theta}) + a_2(\text{ang vel}) + a_3(\text{ang acc}) + a_4$)

(July 5 to July 15)

- **GOALS:**

- ❖ Set Brake Current while retrieving RPM values, etc, simultaneously.
- ❖ Easy user interface for gain values that are communicated with the espidf code.
- ❖ Need provisions for multiple VESCs to be integrated (for both wheels of the rig).
- ❖ Attempt in esp idf(C), to integrate with uROS when ESP-IDF + uROS issues are fixed.

- VESC communication:

ISSUES:

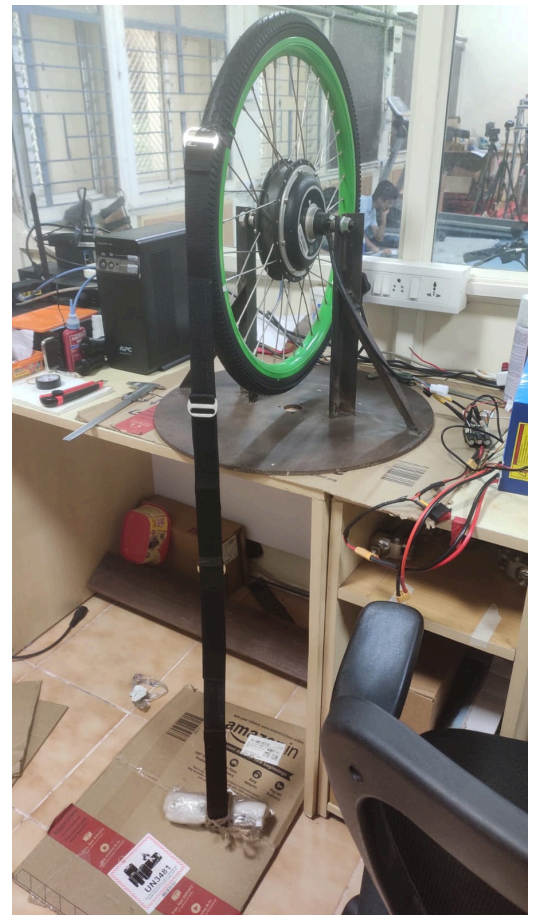
- (i) When the VESC's TX is involved with giving measured values: RPM, position, etc, and RX is receiving the command to set brake current- The serial port seems to be overloaded. A delay between loop iterations helps a bit but as a result, brake current is stopped during these delays causing jarring.
- (ii) RPM values show the same sign regardless of rotation direction when a current is set.
- (iii) The SetBrakeCurrent function hence does not work since it makes use of the RPM measured.
- (iv) There is very less time to observe the effect of a set current at lowloads- since rpm ramps up very quick and the motor has a limit of 5000 rpm.

SOLUTION:

Creating our own setBrakeCurrent block by using setCurrent, since setCurrent works on its own for the VESC, and altering its value based on the rpm retrieved separately. An encoder can be used to receive RPM and position data and fed to the ESP.

- Motor Torque Constant:

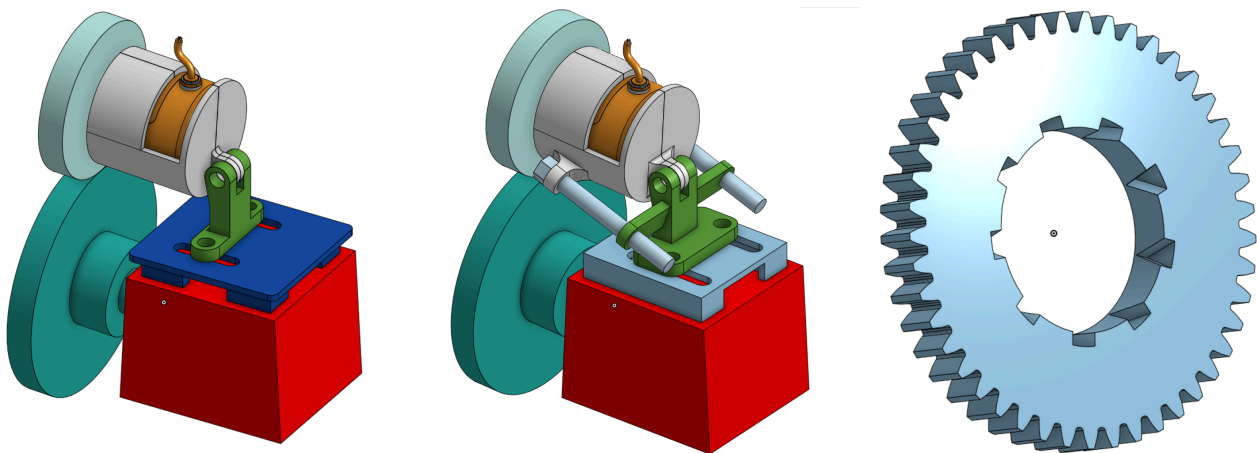
Created a simple setup to carry a load and identify the current required to maintain the hanging load. It takes 1.3A for ~1.2kg(/load torque of 3.58Nm)
An average of such readings can give us a rough torque constant.



4. Encoder Mount and Coupling for existing XRIM Test Rig and Testing Data Retrieval

(July 16 to July 27)

- There is a requirement for accurate RPM and position values to feed to VESC for good current/torque control, and hence an encoder mount for the current setup.
- Some major considerations while designing the mount and coupling:
 - To prevent partial Gear Meshing:
Provided extra DOF for measured and assembly inaccuracies in CAD and varying dimensions of the rig on either side of it.



- To prevent excess play in the DOFs provided by some external tensioning.
- **RESULTS:**
Successfully 3D printed the model and assembled on the test rig. Encoder values for RPM were received by the VESC and the VESC was able to setCurrent based on what it had received.
- **ISSUES & FURTHER STEPS:**
Unfortunately, even the encoder RPM readings were inaccurate. It is necessary to try out the other available encoders. Once an accurate RPM value is obtained, setCurrent can be easily implemented through the VESC itself, and an interface for the brake current function-
(BrakeCurrent = $a_1(\text{theta} - \text{mean theta}) + a_2(\text{ang vel}) + a_3(\text{ang acc}) + a_4$)
can be developed.

5. Additional Documentation:

1. [Commands, Resources for Easy Use of ROS2, Gazebo, uROS, ESP-IDF, VESC tool](#)
2. [Kadham Knee Pin Surface Quality Documentation](#)
3. [Work Completed \(Zip Files\)](#)